The uv build backend

A build backend transforms a source tree (i.e., a directory) into a source distribution or a wheel.

uv supports all build backends (as specified by PEP 517), but also provides a native build backend (`uv_build`) that integrates tightly with uv to improve performance and user experience.

Choosing a build backend

The uv build backend is a great choice for most Python projects. It has reasonable defaults, with the goal of requiring zero configuration for most users, but provides flexible configuration to accommodate most Python project structures. It integrates tightly with uv, to improve messaging and user experience. It validates project metadata and structures, preventing common mistakes. And, finally, it's very fast.

The uv build backend currently **only supports pure Python code**. An alternative backend is required to build a library with extension modules.

!!! tip

```
While the backend supports a number of options for configuring your project
structure, when build scripts or
a more flexible project layout are required, consider using the
[hatchling](https://hatch.pypa.io/latest/config/build/#build-system) build backend
instead.
```

Using the uv build backend

To use uv as a build backend in an existing project, add `uv_build` to the `[build-system]` section in your `pyproject.toml`:

```toml
[build-system]
requires = ["uv_build>=0.12.7,<0.13"]
build-backend = "uv_build"
```

!!! note

```
The uv build backend follows the same [versioning policy](../reference/policies/
versioning.md)
as uv. Including an upper bound on the `uv_build` version ensures that your package
continues to
build correctly as new versions are released.
```

To create a new project that uses the uv build backend, use `uv init`:

```
$ uv init
```

When the project is built, e.g., with `uv build`, the uv build backend will be used to create the source distribution and wheel.

Bundled build backend

The build backend is published as a separate package (`uv_build`) that is optimized for portability and small binary size. However, the `uv` executable also includes a copy of the build backend, which will be used during builds performed by uv, e.g., during `uv build`, if its version is compatible with the `uv_build` requirement. If it's not compatible, a compatible version of the `uv_build` package will be used. Other build frontends, such as `python -m build`, will always use the `uv_build` package, typically choosing the latest compatible version.

Modules

Python packages are expected to contain one or more Python modules, which are directories containing an `__init__.py`. By default, a single root module is expected at `src/<package_name>/__init__.py`.

For example, the structure for a project named `foo` would be:

```
pyproject.toml
src
└── foo
    └── __init__.py
```

uv normalizes the package name to determine the default module name: the package name is lowercased and dots and dashes are replaced with underscores, e.g., `Foo-Bar` would be converted to `foo_bar`.

The `src/` directory is the default directory for module discovery.

These defaults can be changed with the `module-name` and `module-root` settings. For example, to use a `FOO` module in the root directory, as in the project structure:

```
pyproject.toml
FOO
└── __init__.py
```

The correct build configuration would be:

```toml
[tool.uv.build-backend]
module-name = "FOO"
module-root = ""
```

Namespace packages

Namespace packages are intended for use-cases where multiple packages write modules into a shared namespace.

Namespace package modules are identified by a `.` in the `module-name`. For example, to package the module `bar` in the shared namespace `foo`, the project structure would be:

```
pyproject.toml
src
└── foo
    └── bar
        └── __init__.py
```

And the `module-name` configuration would be:

```toml
[tool.uv.build-backend]
module-name = "foo.bar"
```

!!! important

`The `__init__.py` file is not included in `foo`, since it's the shared namespace module.`

It's also possible to have a complex namespace package with more than one root module, e.g., with the project structure:

```
pyproject.toml
src
├── foo
```

```
|   └── __init__.py
└── bar
    └── __init__.py
```

While we do not recommend this structure (i.e., you should use a workspace with multiple packages instead), it is supported by setting `module-name` to a list of names:

```toml
[tool.uv.build-backend]
module-name = ["foo", "bar"]
```

For packages with many modules or complex namespaces, the `namespace = true` option can be used to avoid explicitly declaring each module name, e.g.:

```toml
[tool.uv.build-backend]
namespace = true
```

!!! warning

```
Using `namespace = true` disables safety checks. Using an explicit list of module
names is
strongly recommended outside of legacy projects.
```

The `namespace` option can also be used with `module-name` to explicitly declare the root, e.g., for the project structure:

```
pyproject.toml
src
└── foo
    ├── bar
    │   └── __init__.py
    └── baz
        └── __init__.py
```

The recommended configuration would be:

```toml
[tool.uv.build-backend]
module-name = "foo"
namespace = true
```

Stub packages

The build backend also supports building type stub packages, which are identified by the `-stubs` suffix on the package or module name, e.g., `foo-stubs`. The module name for type stub packages must end in `-stubs`, so uv will not normalize the `-` to an underscore. Additionally, uv will search for a `__init__.pyi` file. For example, the project structure would be:

```
pyproject.toml
src
└── foo-stubs
    └── __init__.pyi
```

Type stub modules are also supported for namespace packages.

File inclusion and exclusion

The build backend is responsible for determining which files in a source tree should be packaged into the distributions.

To determine which files to include in a source distribution, uv first adds the included files and directories, then removes the excluded files and directories. This means that exclusions always take precedence over inclusions.

By default, uv excludes `__pycache__`, `*.pyc`, and `*.pyo`.

When building a source distribution, the following files and directories are included:

- The `pyproject.toml`. If uv detects TOML 1.1-only syntax, it issues a warning and automatically enables the `toml-backwards-compatibility` preview feature: the `pyproject.toml` is reformatted for backwards compatibility, and the original file is preserved as `pyproject.toml.orig`. Pass `--preview-feature toml-backwards-compatibility` to enable the feature explicitly and suppress the warning.
- The module under `tool.uv.build-backend.module-root`.
- The files referenced by `project.license-files` and `project.readme`.
- All directories under `tool.uv.build-backend.data`.
- All files matching patterns from `tool.uv.build-backend.source-include`.

From these, items matching `tool.uv.build-backend.source-exclude` and the default excludes are removed.

When building a wheel, the following files and directories are included:

- The module under `tool.uv.build-backend.module-root`
- The files referenced by `project.license-files`, which are copied into the `.dist-info` directory.
- The `project.readme`, which is copied into the project metadata.
- All directories under `tool.uv.build-backend.data`, which are copied into the `.data` directory.

From these, `tool.uv.build-backend.source-exclude`, `tool.uv.build-backend.wheel-exclude` and the default excludes are removed. The source dist excludes are applied to avoid source tree to wheel builds including more files than source tree to source distribution to wheel build.

There are no specific wheel includes. There must only be one top level module, and all data files must either be under the module root or in the appropriate data directory. Most packages store small data in the module root alongside the source code.

!!! tip

```
When using the uv build backend through a frontend that is not uv, such as pip or
`python -m build`, debug logging can be enabled through environment variables with
`RUST_LOG=uv=debug` or `RUST_LOG=uv=verbose`. When used through uv, the uv build
backend shares
the verbosity level of uv.
```

Include and exclude syntax

Includes are anchored, which means that `pyproject.toml` includes only `<root>/pyproject.toml` and not `<root>/bar/pyproject.toml`. To recursively include all files under a directory, use a `/**` suffix, e.g. `src/**`. Recursive inclusions are also anchored, e.g., `assets/**/sample.csv` includes all `sample.csv` files in `<root>/assets` or any of its children.

!!! note

```
For performance and reproducibility, avoid patterns without an anchor such as `**/
sample.csv`.
```

Excludes are not anchored, which means that `__pycache__` excludes all directories named `__pycache__` regardless of its parent directory. All children of an exclusion are excluded as well. To anchor a directory, use a `/` prefix, e.g., `/dist` will exclude only `<root>/dist`.

All fields accepting patterns use the reduced portable glob syntax from PEP 639, with the addition that characters can be escaped with a backslash.